

The Life-Changing Magic of SQL Server Column Comments



Boost database clarity with SQL Server column comments.

Master the adding of column comments in SQL Server through SSMS for easy database documentation. Step-by-step guide, with code, visuals, and notes on creating clear, maintainable schemas.

Ever stared at a SQL Server table, clueless as to what each column meant, cursing the absence of documentation?

I have been there-wandering all alone in a maze of cryptic column names until I discovered the power of SQL Server column comments.

This article will be your lifeline through adding, removing, and intellectualizing column comments in SSMS so that your database explains itself. Equipped with code snippets, diagrams, and kernels of practical advice, I will share my hard-earned wisdom and hopefully spare you the same documentation despair. Let's bring clarity to your database chaos.

Overview

-  [Why Column Comments Matter](#)
-  [Set Up Your SQL Server Database](#)
-  [Add Column Comments in SSMS](#)
-  [Query Column Comments for Reference](#)
-  [Update and Maintain Comments](#)
-  [Use Comments in Database Design](#)
-  [Share Comments with Your Team](#)
-  [Best Practices for Comment Management](#)

Why Column Comments Matter

Column comments in SQL Server detail the purpose of columns, making it easier to understand the database. I learned this after spending hours deciphering mystery columns.

Additional comments on a column are metadata residing in SQL Server's extended properties to describe the column's role (e.g., "Customer e-mail address"). They improve:

- 🧑💻 **Readability:** Explain the purpose of the columns to the developers.
- 🛠️ **Maintenance:** Reduced updates and debugging.
- 💛 **Collaboration:** Assist the team members in making them know about the schema.

Use comment features to design self-documenting databases rather than linking to external documents. The comments can be viewed through SQL Server Management Studio (SSMS) or by executing SQL queries, whereas external wikis tend to become outdated.



Understanding Table Components

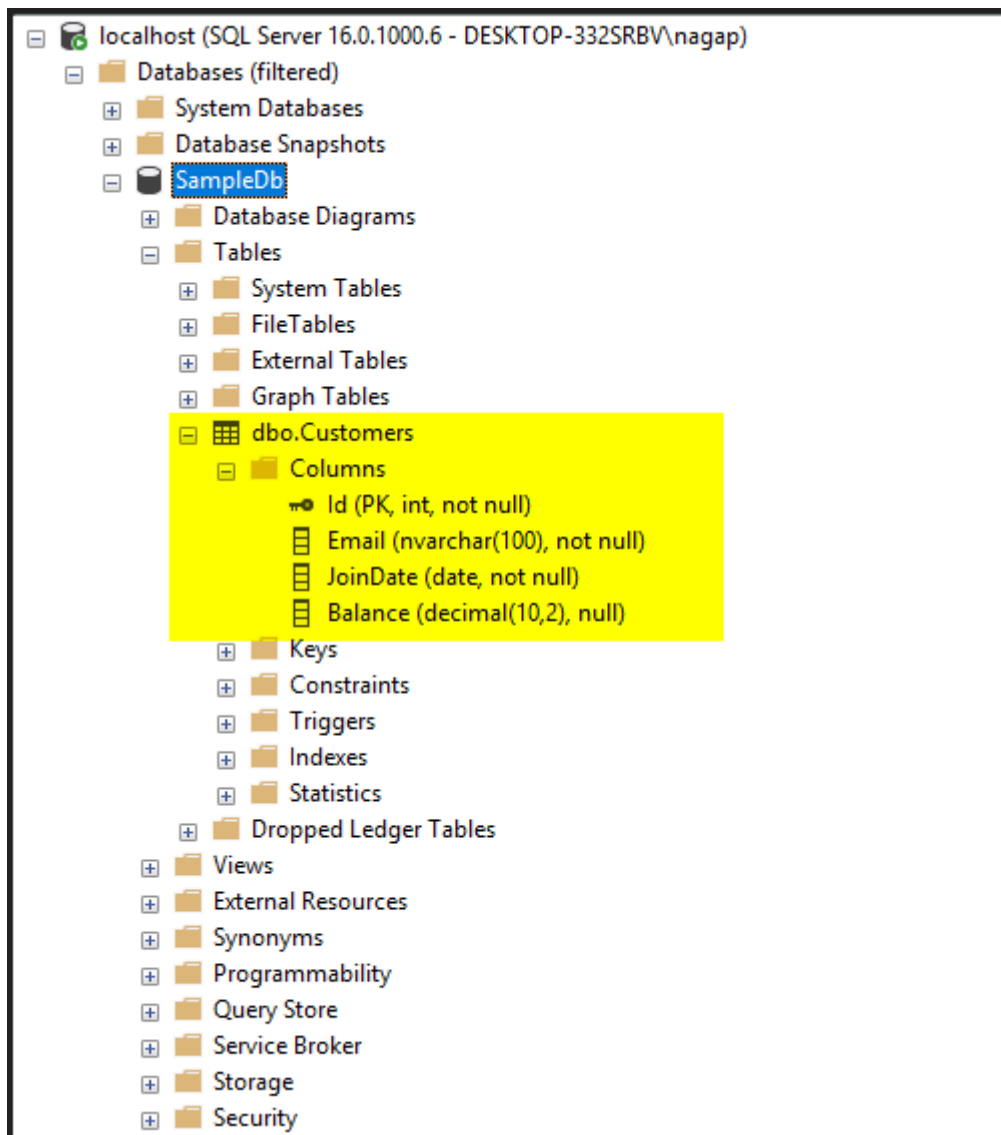
Complex columns are converted into meaningful pieces of information by comments placed inside the cryptic scripts.

Set Up Your SQL Server Database

A well-structured database is the foundation for meaningful commenting. Messy schemas have troubled me until I streamlined this.

● In SSMS, create a database and a table:

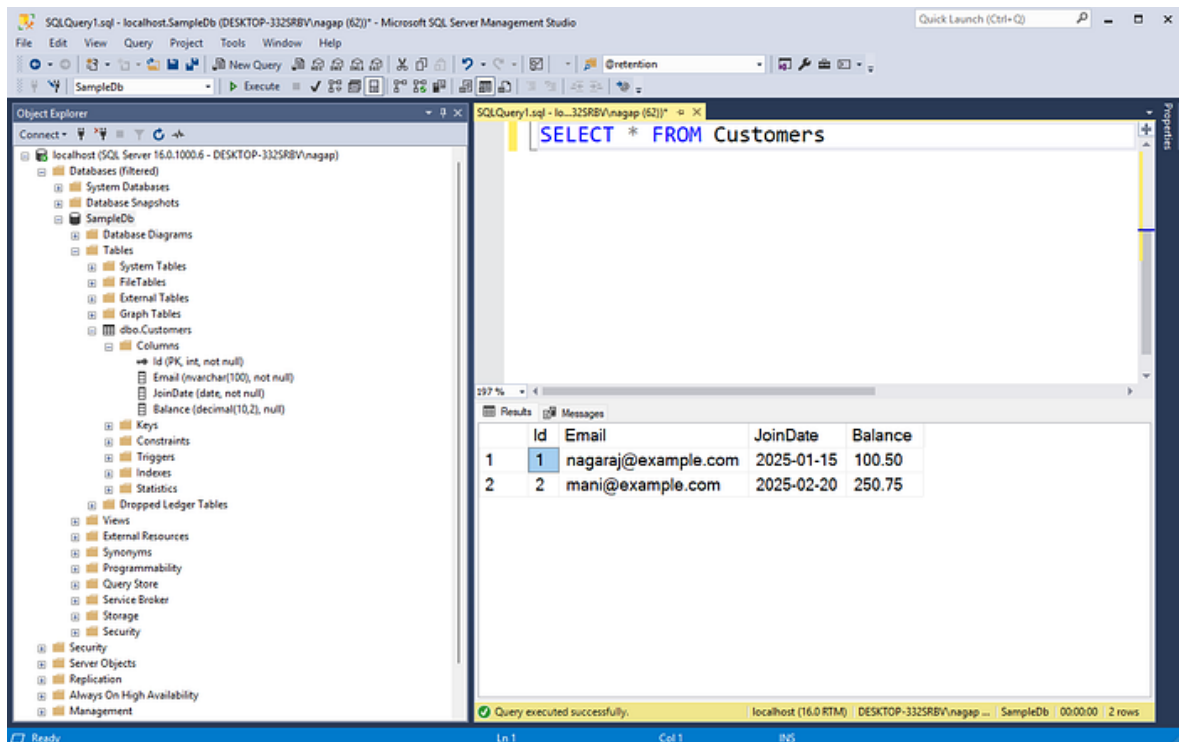
```
CREATE DATABASE SampleDb;
GO
USE SampleDb;
GO
CREATE TABLE Customers (
    Id INT PRIMARY KEY IDENTITY(1,1),
    Email NVARCHAR(100) NOT NULL,
    JoinDate DATE NOT NULL,
    Balance DECIMAL(10,2) DEFAULT 0.00
);
```



● Insert sample data:

```
INSERT INTO Customers (Email, JoinDate, Balance) VALUES
('nagaraj@example.com', '2025-01-15', 100.50),
('mani@example.com', '2025-02-20', 250.75);
```

📌 You are required to follow this way of creating the Customers table in the SampleDb database so as to add comments. Ensure that your SQL Server instance is connected.



A good database setup encourages clear discussion.

Add Column Comments in SSMS

Adding column comments to SQL Server basically means a phrase that describes the column, and these comments are good for documentation. I used to skip this step, making maintenance difficult.

● The **sp_addextendedproperty** stored process has to be used:

```
EXEC sp_addextendedproperty
    @name = N'MS_Description',
    @value = N'Unique customer identifier',
    @level0type = N'SHEMA', @level0name = 'dbo',
    @level1type = N'TABLE', @level1name = 'Customers',
    @level2type = N'COLUMN', @level2name = 'Id';
```

```
EXEC sp_addextendedproperty
    @name = N'MS_Description',
    @value = N'Customer email address',
    @level0type = N'SHEMA', @level0name = 'dbo',
    @level1type = N'TABLE', @level1name = 'Customers',
    @level2type = N'COLUMN', @level2name = 'Email';
```

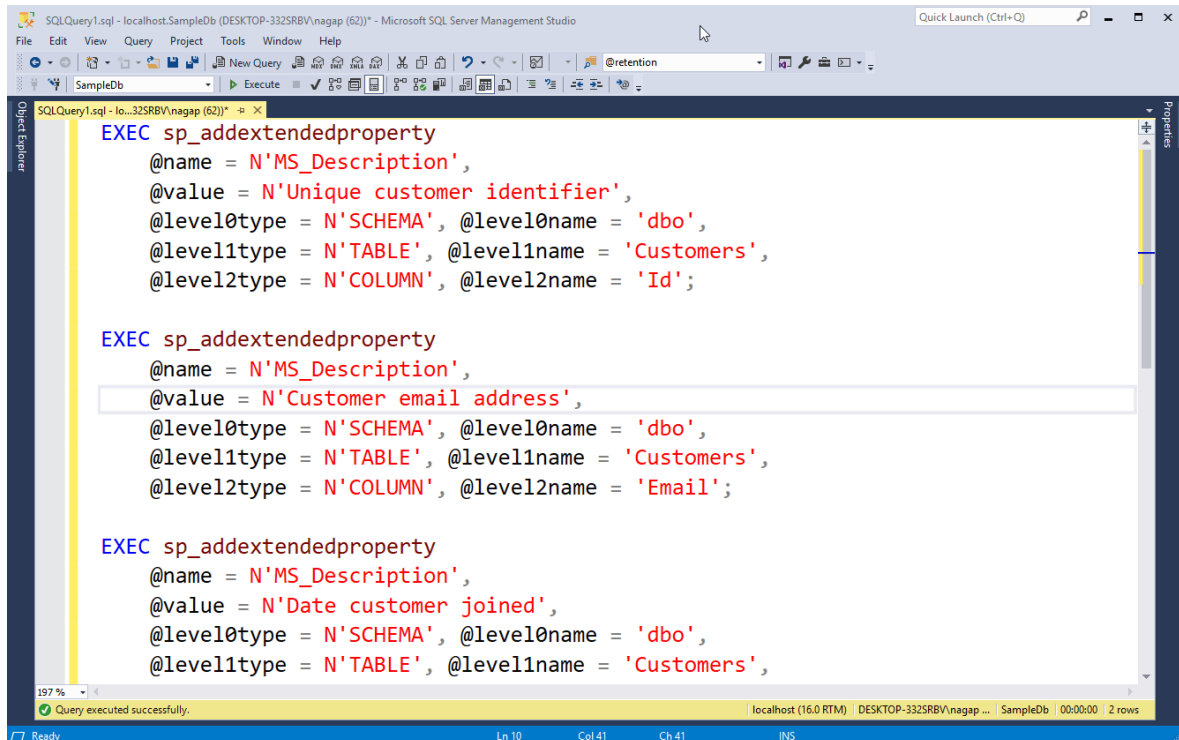
```
EXEC sp_addextendedproperty
    @name = N'MS_Description',
    @value = N'Date customer joined',
    @level0type = N'SHEMA', @level0name = 'dbo',
    @level1type = N'TABLE', @level1name = 'Customers',
    @level2type = N'COLUMN', @level2name = 'JoinDate';
```

```
EXEC sp_addextendedproperty
    @name = N'MS_Description',
    @value = N'Customer account balance in USD',
    @level0type = N'SHEMA', @level0name = 'dbo',
    @level1type = N'TABLE', @level1name = 'Customers',
    @level2type = N'COLUMN', @level2name = 'Balance';
```

Key points:

- **MS_Description:** A standard name for comments.
- **Hierarchy:** State the database, table, and column.
- **Clear descriptions:** Use meaningful, brief text.

Execute these in SSMS to append comments to the Customers table columns.



Comments will get your columns to be self-explanatory.

Query Column Comments for Reference

Querying comments allows working with documentation programmatically. Therefore, I learned this thing to avoid digging into SSMS manually.

● Fetch comments using `sys.extended_properties`:

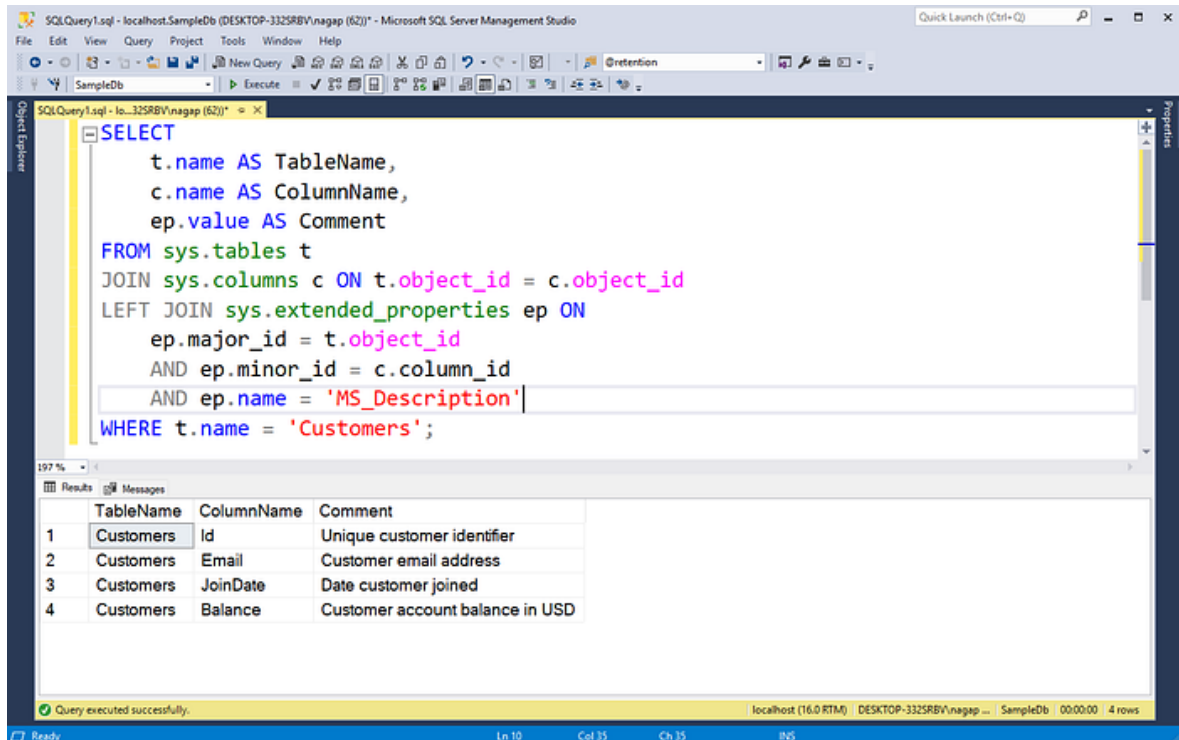
```
SELECT
    t.name AS TableName,
    c.name AS ColumnName,
    ep.value AS Comment
FROM sys.tables t
JOIN sys.columns c ON t.object_id = c.object_id
LEFT JOIN sys.extended_properties ep ON
    ep.major_id = t.object_id
    AND ep.minor_id = c.column_id
    AND ep.name = 'MS_Description'
WHERE t.name = 'Customers';
```

This query retrieves:

- **TableName:** Customers
- **ColumnName:** Id, Email, JoinDate, Balance

- **Comment:** The descriptions were added before.

👉 This can be used to generate documentation or verify comments. Save this query in SSMS to have instant use.



The screenshot shows the Microsoft SQL Server Enterprise Edition interface. The query editor displays the following SQL query:

```
SELECT
    t.name AS TableName,
    c.name AS ColumnName,
    ep.value AS Comment
FROM sys.tables t
JOIN sys.columns c ON t.object_id = c.object_id
LEFT JOIN sys.extended_properties ep ON
    ep.major_id = t.object_id
    AND ep.minor_id = c.column_id
    AND ep.name = 'MS_Description'
WHERE t.name = 'Customers';
```

The Results pane shows the output of the query as a table with 4 rows and 3 columns: TableName, ColumnName, and Comment.

	TableName	ColumnName	Comment
1	Customers	Id	Unique customer identifier
2	Customers	Email	Customer email address
3	Customers	JoinDate	Date customer joined
4	Customers	Balance	Customer account balance in USD

The status bar at the bottom indicates the query was executed successfully on localhost (16.0 RTM) in SampleDb, returning 4 rows.

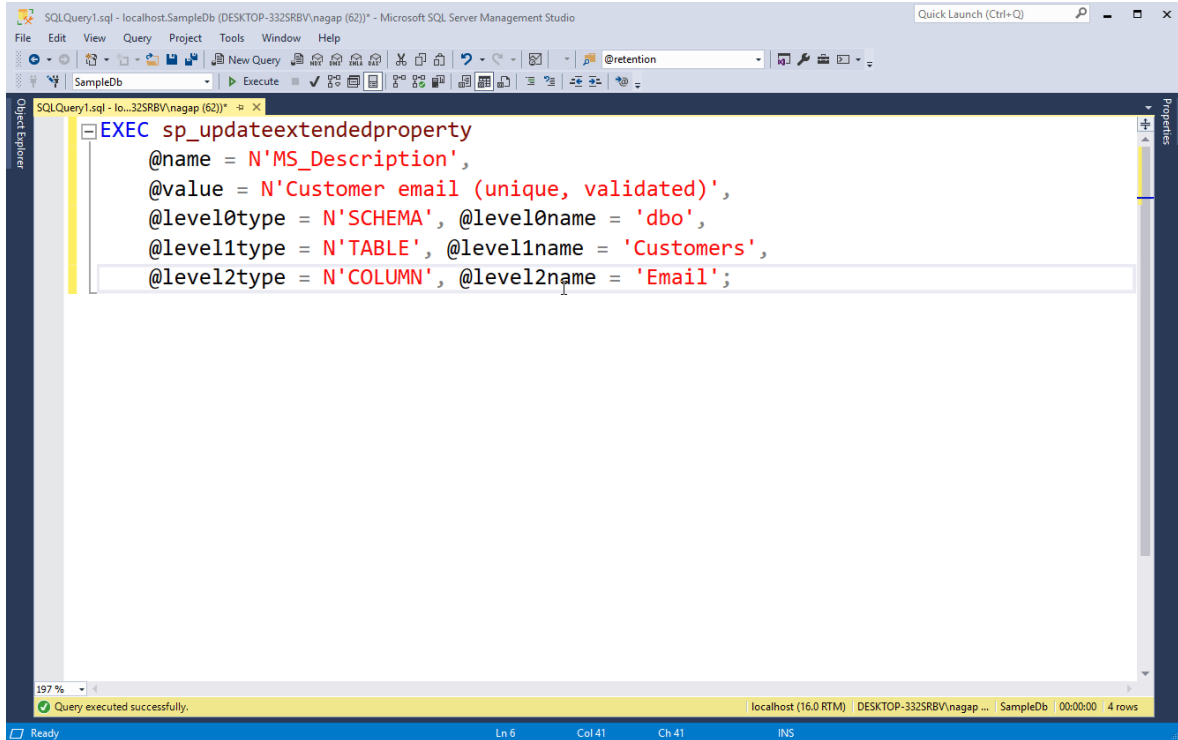
Queries for comments are transferred into instant documentation.

Update and Maintain Comments

Keeping comments up-to-date ensures lasting clarity. I neglected this task only once and paid heavily for this in misunderstanding.

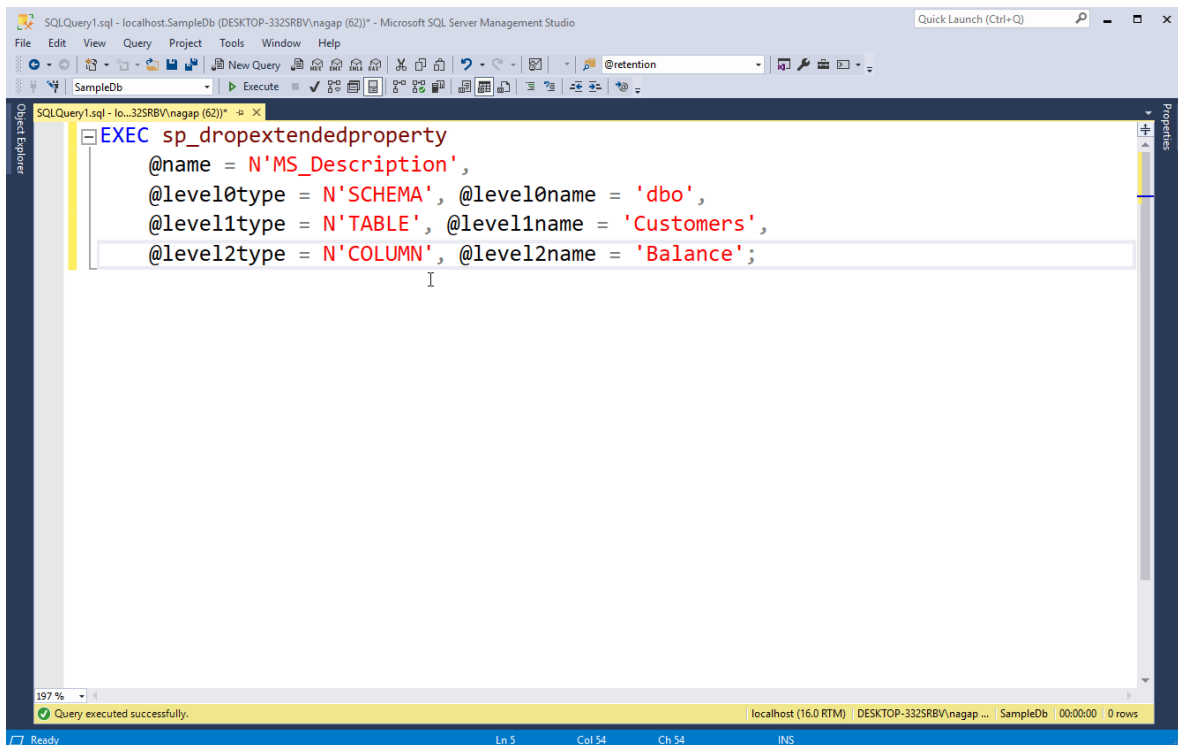
● **The comments may have to be updated by using `sp_updateextendedproperty` :**

```
EXEC sp_updateextendedproperty
    @name = N'MS_Description',
    @value = N'Customer email (unique, validated)',
    @level0type = N'SHEMA', @level0name = 'dbo',
    @level1type = N'TABLE', @level1name = 'Customers',
    @level2type = N'COLUMN', @level2name = 'Email';
```



● Delete comments as the need may be:

```
EXEC sp_dropextendedproperty
    @name = N'MS_Description',
    @level0type = N'SHEMA', @level0name = 'dbo',
    @level1type = N'TABLE', @level1name = 'Customers',
    @level2type = N'COLUMN', @level2name = 'Balance';
```



Maintenance Tips:

- **Review regularly:** Update comments, during any schema changes.
- **Standardize format:** Use consistent wording (e.g., "Stores...").
- **Script updates:** Store your scripts in a version control system.

| Adding Comments Regularly Aids Maintaining Neat Documentations

Use Comments in Database Design

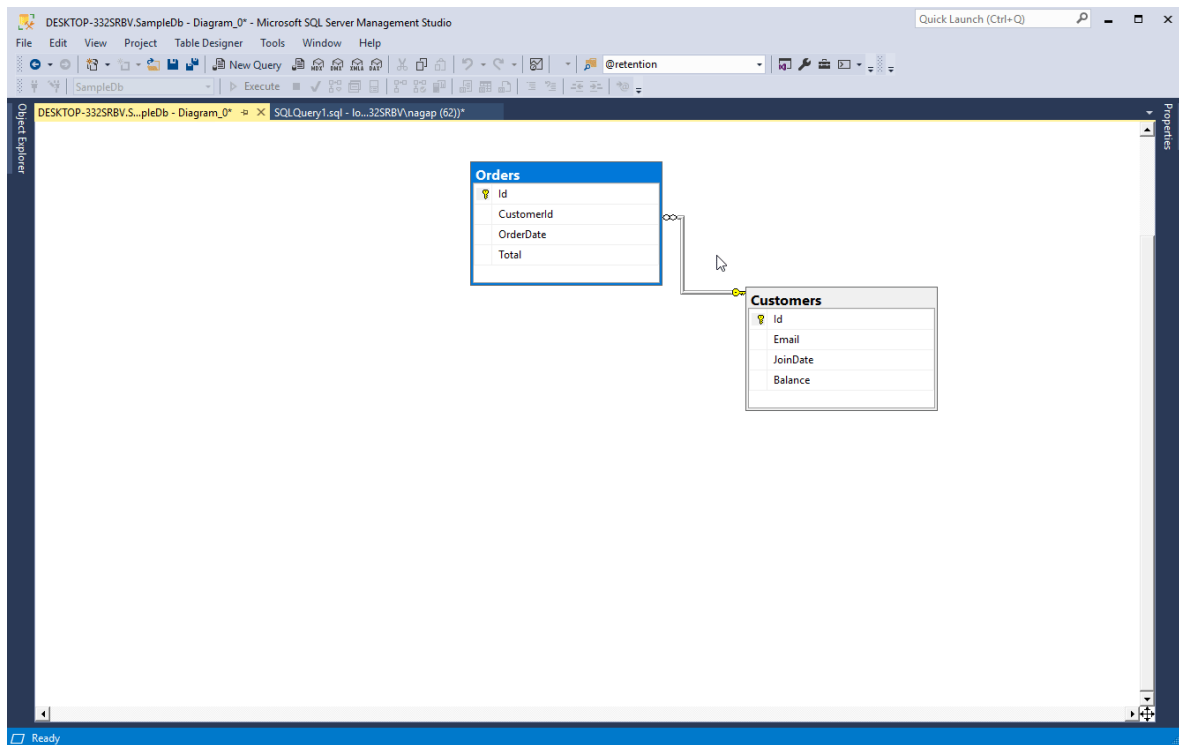
Comments make database design better by showing the purposefulness of the designer. I could also design anything until I started commenting inserts and procedures where my intricate methodology became clear.

● Add comments during table creation:

```
CREATE TABLE Orders (  
    Id INT PRIMARY KEY IDENTITY(1,1),  
    CustomerId INT NOT NULL,  
    OrderDate DATETIME NOT NULL,  
    Total DECIMAL(10,2) NOT NULL,  
    FOREIGN KEY (CustomerId) REFERENCES Customers(Id)  
);  
EXEC sp_addextendedproperty  
    @name = N'MS_Description',  
    @value = N'Unique order identifier',  
    @level0type = N'SHEMA', @level0name = 'dbo',  
    @level1type = N'TABLE', @level1name = 'Orders',  
    @level2type = N'COLUMN', @level2name = 'Id';  
EXEC sp_addextendedproperty  
    @name = N'MS_Description',  
    @value = N'Links to customer in Customers table',  
    @level0type = N'SHEMA', @level0name = 'dbo',  
    @level1type = N'TABLE', @level1name = 'Orders',  
    @level2type = N'COLUMN', @level2name = 'CustomerId';
```

Design tips:

- **Document relationships:** Point out foreign keys (for example CustomerId).
- **Clarify constraints:** Please explain defaults or procedures.
- **Plan ahead:** Add comments during design than afterward.



Comments provide sufficient clarity for your design.

Share Comments with Your Team

Sharing comments helps document so that your team benefits. I learned this after my teammates sweated over my tables.

● Generate report comments:

```
SELECT
    t.name AS TableName,
    c.name AS ColumnName,
    ep.value AS Comment
FROM sys.tables t
JOIN sys.columns c ON t.object_id = c.object_id
LEFT JOIN sys.extended_properties ep ON
    ep.major_id = t.object_id
    AND ep.minor_id = c.column_id
    AND ep.name = 'MS_Description'
WHERE t.schema_id = SCHEMA_ID('dbo')
ORDER BY t.name, c.name;
```

Sharing Techniques:

- **Export to Excel:** Please run the script in SSMS and export the results using CSV.

- **Integrate with docs:** Include in team wikis or READMEs.
- **Automate reports:** Schedule queries using SQL Server Agent.

The screenshot shows the Microsoft SQL Server Management Studio interface. The main window displays a SQL query that retrieves table and column comments from the 'SampleDb' database. The query uses a combination of JOIN and LEFT JOIN to fetch data from sys.tables, sys.columns, and sys.extended_properties. The results are displayed in a table below the query editor.

```
SELECT
    t.name AS TableName,
    c.name AS ColumnName,
    ep.value AS Comment
FROM sys.tables t
JOIN sys.columns c ON t.object_id = c.object_id
LEFT JOIN sys.extended_properties ep ON
    ep.major_id = t.object_id
    AND ep.minor_id = c.column_id
    AND ep.name = 'MS_Description'
WHERE t.schema_id = SCHEMA_ID('dbo')
ORDER BY t.name, c.name;
```

	TableName	ColumnName	Comment
1	Customers	Balance	NULL
2	Customers	Email	Customer email (unique, validated)
3	Customers	Id	Unique customer identifier
4	Customers	JoinDate	Date customer joined
5	Orders	CustomerId	Links to customer in Customers table
6	Orders	Id	Unique order identifier
7	Orders	OrderDate	NULL
8	Orders	Total	NULL

Query executed successfully. Status bar: localhost (16.0 RTM) | DESKTOP-332SRBV\nagap | SampleDb | 00:00:00 | 13 rows

Send the results into a shared drive or circulate via emails within members.

Best Practices for Comment Management

The efficient management of comments keeps the database clean and available for use. I refined these processes after a few cases of documentation getting sloppy and chaotic.

👉 *Follow this few steps now:*

- **Be concise:** Use short and clear explanations (e.g., "Customer email, unique").
- **Use consistent naming:** Go with `MS_Description` whenever you need the comments.
- **Version control:** Keep comment scripts in Git.
- **Audit periodically:** Please look out for outdated or missing comments.

● **Run this script to retrieve any missing comments:**

```

SELECT
    t.name AS TableName,
    c.name AS ColumnName
FROM sys.tables t
JOIN sys.columns c ON t.object_id = c.object_id
LEFT JOIN sys.extended_properties ep ON
    ep.major_id = t.object_id
    AND ep.minor_id = c.column_id
    AND ep.name = 'MS_Description'
WHERE ep.value IS NULL
    AND t.schema_id = SCHEMA_ID('dbo');

```

Ensure that every cabinet is documented by running a query consistently.

Comment discipline will make your database proceed favorably.

Final notes

- **Value comments:** Clarify the purpose of the column for better understanding.
- **Set up database:** Tables are set up with sample data.
- **Add comments:** Use `sp_addextendedproperty` in SSMS.
- **Query comments:** Access metadata via `sys.extended_properties`.
- **Maintain comments:** Update or delete using stored procedures.
- **Design with comments:** Record during schema creation.
- **Share with team:** Export the comments to collaborate.
- **Follow best practices:** Please remember to keep comments short and to the point.